



Data laag en chat-kennis voor Optimax

Onderzoek + aanbeveling, uitgelegd voor niet-specialisten. 11 juni 2026. Gebaseerd op drie parallelle onderzoeken (PostgreSQL-vertaallaag, Grafana/chat-integratie, fabriekskennis voor de chat).

Waar dit over gaat

Optimax verwacht nette, begrijpelijke tabellen (alarmen, productie-tellers per minuut, palletstatus). Maar de echte database op de logger bevat **ruwe PLC-tabellen** met namen als `readconfig2` en `test024`, gevuld door Node-RED met metingen tot 4x per seconde. Er moet dus iets tussen: een **vertaallaag** die van ruwe metingen nette cijfers maakt.

Die vertaallaag krijgt straks **drie afnemers** die alle drie hetzelfde moeten zien: 1. **Het Optimax-dashboard** (het klant-product), 2. **Grafana** (het interne gereedschap dat al op de logger draait), 3. **De AI-chat** (die zelf databasevragen schrijft).

En de chat heeft nog iets extra's nodig: **fabriekskennis**. Niet alleen "welke tabellen bestaan er", maar "hoe ziet de lijn er fysiek uit, wat meet welke sensor, en hoe hangt alles samen". Dit document legt uit wat we onderzochten, wat eruit kwam, en wat het plan is.

1. De begrippen eerst (in gewone taal)

Begrip	In gewone taal
View	Een opgeslagen bril : je kijkt door de bril naar de ruwe tabel en ziet nette kolommen met begrijpelijke namen. Er wordt niets gekopieerd; de bril vertaalt live bij elke vraag.
Materialized view	Een foto van wat je door die bril ziet. Sneller om naar te kijken (alles is al uitgerekend), maar de foto verouderd: je moet hem regelmatig opnieuw maken ("verversen").
Partitionering	De ruwe tabellen zijn opgeknipt in één lade per dag . Wie met een datum vraagt, hoeft maar één lade open te trekken in plaats van alle 90.
COMMENT ON	Post-its op de database zelf : bij elke tabel en kolom kun je in de database een uitleg plakken ("counter0 = teller lijn 1, stuks per minuut"). Mensen én de AI kunnen die post-its lezen.
Introspectie	De database over zichzelf laten vertellen : een programma vraagt "welke tabellen en kolommen heb jij, en wat staat er op de post-its?" en krijgt dat live terug.
Text-to-SQL	Wat de chat doet: jouw vraag in gewone taal omzetten naar een databasevraag (SQL).
Semantische laag	Het geheel van brillen + post-its: een begrijpelijke buitenkant over de ruwe data heen, zodat mens en AI nooit met de rommelige binnenkant hoeven te praten.
Topologie	De plattegrond van de lijn : welke baan voedt welke robot, waar liggen de palletstations, welke sensor meet wat.
eMMC / flash-slijtage	De opslag van de logger is een soort grote SD-kaart die slijt van schrijven . Onnodig veel kleine schrijfacties verkorten zijn leven.

2. De vertaallaag: hoe maken we van ruwe data nette tabellen?

We vergeleken vijf manieren. Samengevat:

Optie	In gewone taal	Oordeel
1. Gewone views	De opgeslagen bril: vertaalt live	Hiermee starten. Nul extra onderdelen, niets dat kan vastlopen. Snel genoeg zolang vragen een datum/tijdvenster meegeven (en dat doet Optimax altijd).
2. Materialized views	De foto, elke paar minuten ververs	Opschaal-stap zodra een grafiek-vraag structureel langer dan ~1 seconde duurt. Verversen kan via het al aanwezige Node-RED (elke minuut een "ververs"-opdracht).
3. Aggregatie bij het schrijven	Node-RED of de database rekent bij elke meting meteen het minuutcijfer bij	Afgeraden. Bij 4 metingen per seconde betekent dit duizenden extra schrijfacties per uur: precies het soort kleine schrijfwerk waar de flash-opslag van slijt (4-8x versterkt).
4. TimescaleDB	Een gespecialiseerde tijdreeksen-uitbreiding die dit alles automatisch doet	Technisch de sterkste, maar het zwaarste middel: vereist een migratie van de database op het klant-apparaat waar Node-RED live naartoe schrijft. Alleen overwegen als optie 2 het niet redt.
5. Apart kopieer-proces	Node-RED kopieert periodiek ruwe data naar nette tabellen	Werkbaar, maar lost niets op dat optie 1/2 niet ook oplossen, en maakt Node-RED een extra risicopunt.

De ladder (zo gaan we het doen): 1. **Fase 1, nu:** een apart database-"schema" **optimax** met gewone views, plus een zuinige index (BRIN, ~8 KB in plaats van ~25 MB) op de tijd-kolom van de ruwe tabellen. Het dashboard, Grafana en de chat krijgen een leesrol op alléén dat schema. 2. **Fase 2, bij bewezen traagheid:** de drukst-bevraagde views vervangen door materialized views, ververs door Node-RED. 3. **Fase 3, alleen bij echt volume-probleem:** TimescaleDB, gepland met een onderhoudsvenster bij DGS.

Belangrijk praktisch punt: de view-definities kunnen **nu al geschreven worden** met invul-plekken voor de echte kolomnamen (de DDL-template staat in het bronrapport). Zodra bekend is welke ruwe kolom wat betekent, vullen we ze in; de rest van het systeem merkt er niets van.

3. Drie afnemers, één waarheid

Het gevaar bij drie afnemers: dat "stilstand-minuut" of "piekuur" op drie plekken nét anders gedefinieerd wordt (in de dashboard-code, in een Grafana-paneel, en door de chat). Dan wijken de cijfers af en weet niemand welk getal klopt.

De oplossing: definieer elk kengetal precies één keer, in de database (als view, of als functie wanneer er een instelbare drempel in zit), en laat alle drie de afnemers diezelfde definitie gebruiken. Dashboard leest de view, Grafana leest de view, de chat schrijft zijn vragen tegen de view.

Wat het onderzoek daar verder over vond:

- **Voor de chat is dit het verschil tussen onbruikbaar en betrouwbaar.** Benchmarks (2024-2026): AI-gegenereerde SQL op ruwe tabellen scoort ~40-65% goed; op een nette view-laag 84-100%. Belangrijker nog dan het percentage: het **faalgedrag** verandert. Op ruwe tabellen geeft de AI "vrolijk een verkeerd getal" (stil en plausibel); op een nette laag krijg je een foutmelding als iets niet kan. Voor een fabrieksdashboard is dat cruciaal.
- **De post-its (COMMENT ON) zijn de goedkoopste grote winst:** in onderzoek steeg de nauwkeurigheid van 58% naar 86% door alleen al uitleg bij kolommen te plakken.
- **De hardcoded schema-tekst in de chat kan vervangen worden door introspectie:** de chat vraagt bij het opstarten aan de database zelf "wat heb je en wat betekenen de kolommen?" (inclusief post-its). Dan kunnen database en chat **nooit meer uit de pas lopen**, wat nu een sluimerend risico is.
- **Grafana op views werkt prima**, met drie hardgrenzen zodat een intern Grafana-dashboard de kleine logger nooit kan platleggen: een eigen leesrol met alleen rechten op de views, een maximale querytijd (statement timeout), en limieten op rijen/verbindingen.
- **Grafana blijft intern.** Grafana-panelen in het klant-product embedden kan technisch, maar anonieme toegang is een datalek- én belastingsrisico, en het netjes wegpoetsen van het Grafana-merk is mogelijk een betaalde Enterprise-licentievraag. Optimax is het klantgezicht; Grafana het interne gereedschap.

4. Hoe leert de chat de fabriek kennen?

Dit was jouw kernvraag: hoe weet de chat hoe de lijn eruitziet, wat waar staat en wat waar gemeten wordt? Het onderzoek onderscheidt **drie kennislagen**:

1. **Structuur:** welke tabellen en kolommen bestaan er? (heeft de chat nu al, hardcoded) 2. **Betekenis:** wat betekent een kolom of statuscode in het echt? (ontbreekt grotendeels) 3. **Plattegrond:** hoe hangt alles fysiek samen, welke baan voedt welke robot? (ontbreekt volledig)

Vijf patronen vergeleken (lange systeem-tekst, post-its in de database, een apart kennisdocument met zoekstap, een opvraagbare plattegrond-tool, een volwaardige KPI-laag). De aanbeveling voor onze situatie (één lijn, klein apparaat, Nederlandstalige chat) is een **combinatie van drie lichte dingen**:

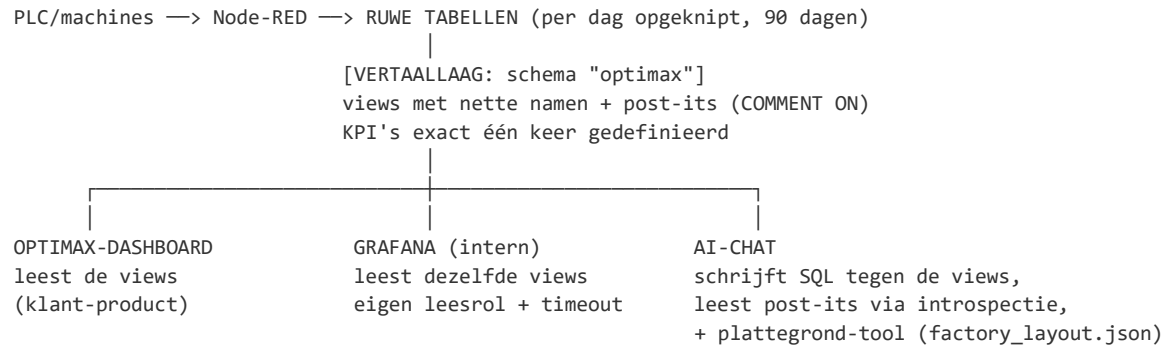
1. **Een plattegrond-bestand + een tool.** De fysieke opstelling komt in één bestand in de repo (`factory_layout.json`): de twee invoerbanen (lijn 2 en 3), de twee robots, de aflegbanen (lijn 1 en 4), het overflow-principe, de palletstations met hun statuscodes, en straks de motoren per lijn. De chat krijgt een extra "telefoonnummer" (function-call `get_factory_layout`) waarmee hij die plattegrond **alleen opvraagt als de vraag erom vraagt**. Een statistiekvraagje kost dan geen extra tokens; een "waarom is lijn 2 zo druk?"-vraag krijgt de volledige samenhang. Wijzigt de lijn (motor erbij, extra teller), dan is dat één bestandsaanpassing. 2. **Post-its in de database** (COMMENT ON) op alle views en kolommen, plus introspectie zodat de chat schema + betekenis altijd actueel binnenkrijgt. 3. **Een compacte basis in de systeem-tekst** (max ~300-400 woorden): shift-tijden, het overflow-principe in twee zinnen, alarm-betekeningen. Niet meer dan dat: onderzoek laat zien dat AI-redeneren verslechtert als de vaste prompt

voorbij ~3000 tokens groeit ("lost in the middle"), dus de volledige plattegrond hoort in de tool, niet in de prompt.

Hoe testen we dat het werkt? Met een vaste set domeinvragen waar maar één goed antwoord op is, bijvoorbeeld: "welke lijn is de aflegplaats van robot 1?" (lijn 1), "wat betekent hoge overflow op lijn 3?" (de robot pakt te weinig weg), "wat betekent statuscode 300 op station 6005?". Het bronrapport bevat een 8-vragen-testset met als lat 7 van de 8 goed. Kosten: verwaarloosbaar (minder dan een tiende cent per chatvraag extra).

Eén kanttekening voor de eerlijkheid: de exacte betekenis van de palletstatuscodes en de robot-toewijzing per station in het voorbeeld-bestand moeten nog **gevalideerd worden met DGS**; het onderzoeksvoorbeeld gokte daar deels (bijv. 200 = " bezig" waar onze code "leeg, wacht op vulling" gebruikt). De plattegrond invullen doen we dus samen met iemand die de lijn kent.

5. De hele plaat bij elkaar



Eén bron van waarheid, drie afnemers, en de chat begrijpt zowel de cijfers (views + post-its) als de fabriek (plattegrond-tool).

6. Concreet stappenplan

#	Stap	Kan al?
1	<code>optimax</code> -schema + view-DDL klaarzetten met invul-plekken voor de echte kolomnamen	Nu (template staat klaar in het bronrapport)
2	<code>factory_layout.json</code> opstellen en de inhoud valideren met DGS (lijn-toewijzing, statuscodes)	Nu opstellen , valideren bij volgend DGS-contact
3	Chat: <code>get_factory_layout</code> -tool + compacte basis-prompt + introspectie van COMMENT ON	Na stap 2 (bouwwerk is klein, past op de bestaande chat)
4	Kolom-mapping ruwe tabel → view invullen	Wacht op DGS : welke ruwe kolom betekent wat
5	Grants verhuizen: dashboard-, chat- en Grafana-rol naar het <code>optimax</code> -schema	Bij stap 4

6	Domeinvragen-testset draaien (8 vragen, lat 7/8)	Na stap 3
7	Meten of views snel genoeg zijn; pas dan eventueel materialized views (fase 2)	Na go-live met echte data

7. Open punten en beslissingen

- **Kolom-mapping onbekend** (welke ruwe kolom = welke teller): blokkeert stap 4-5, niet stap 1-3.
- **Plattegrond valideren met DGS**: statuscodes en robot/station-toewijzing bevestigen.
- **Timezone-vraag** (schrijft Node-RED lokale tijd of UTC?) loopt nog; de 1-minuut-probe staat in TODO.md en bepaalt mede hoe de views met tijd omgaan.
- **Grafana-branding**: als DGS ooit Grafana-panelen in het klant-product wil, eerst de Enterprise-licentievraag bij Grafana Labs checken. Default: Grafana blijft intern.

Bronnen

Dit is de synthese van drie onderzoeksrapporten (in de Uland-monorepo onder [output/research/](#)): [optimax-data laag-postgres-20260611.md](#), [optimax-data laag-grafana-chat-20260611.md](#) en [optimax-chat-fabriekskennis-20260611.md](#), elk met volledige bronvermelding. Kernbronnen: PostgreSQL-documentatie (partitioning, materialized views, COMMENT), Grafana-docs (PostgreSQL-datasource, embedding), dbt-benchmark 2026 (semantische laag vs ruwe text-to-SQL), Tiger Data (COMMENT-effect op LLM-nauwkeurigheid), TimescaleDB-docs, eMMC-slijtage-whitepapers (Kingston, SkyHigh), en arXiv-werk over function-calling en prompt-lengte-degradatie.